

LlamaIndex: 增强私有数据接入大模型

一份面向开发者的LlamaIndex框架入门与实践指南



LlamaIndex

本次分享大纲

01. 什么是LlamaIndex?

- 核心定义与目标

02. LlamaIndex为何而生?

- 大语言模型的固有局限性

03. LlamaIndex如何破局?

- 核心架构与关键组件解析

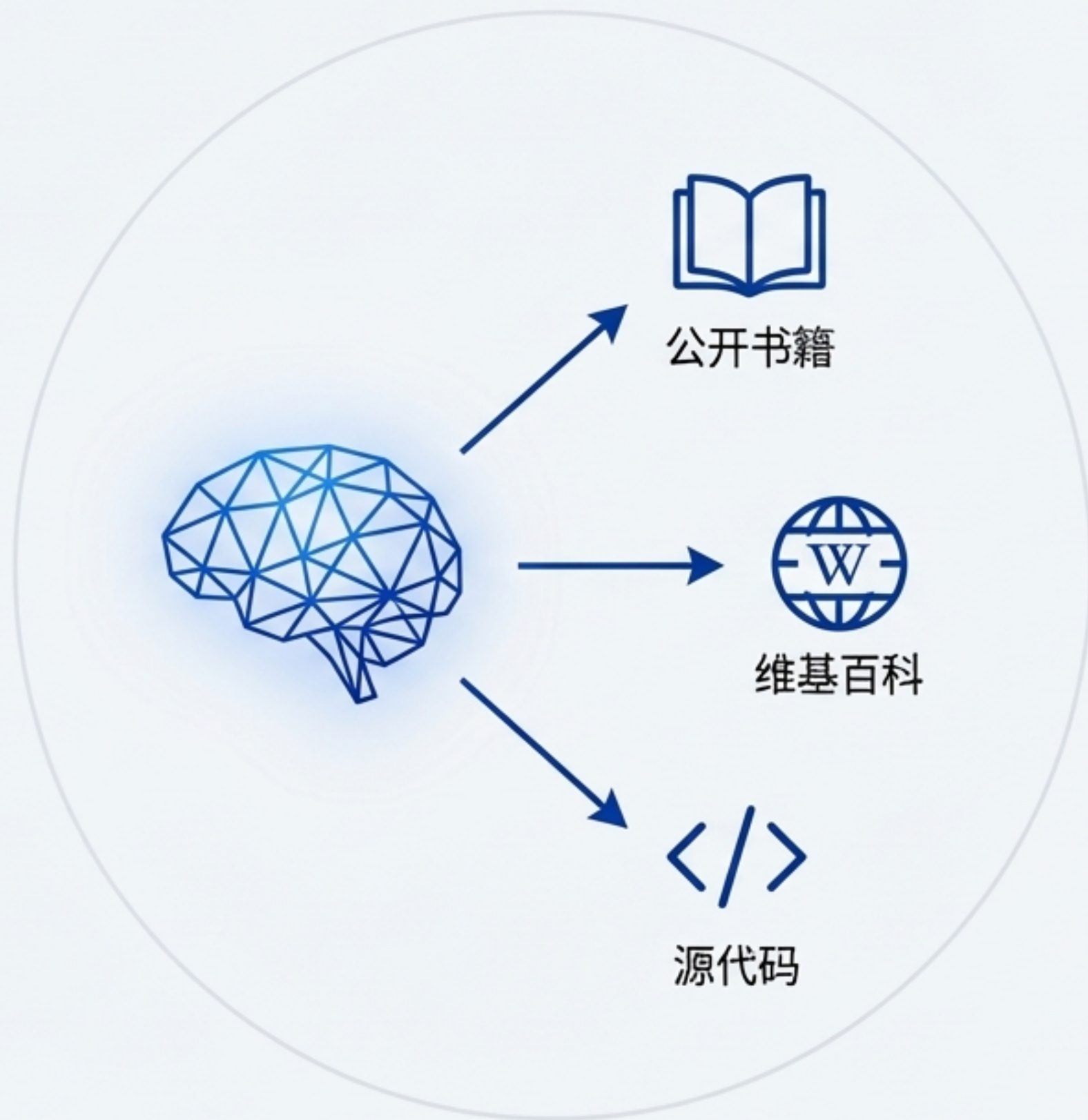
04. LlamaIndex实战应用

- 五步构建一个私有知识库问答机器人

一切的开始： 大语言模型知道什么？

大语言模型（如GPT）的知识来源于其预训练所使用的海量公开数据。

- **知识来源：** 维基百科、公开书籍、邮件列表、源代码等。
- **核心局限：** 它们的知识是静态的，并且无法访问你的私有、实时或特定领域的数据。



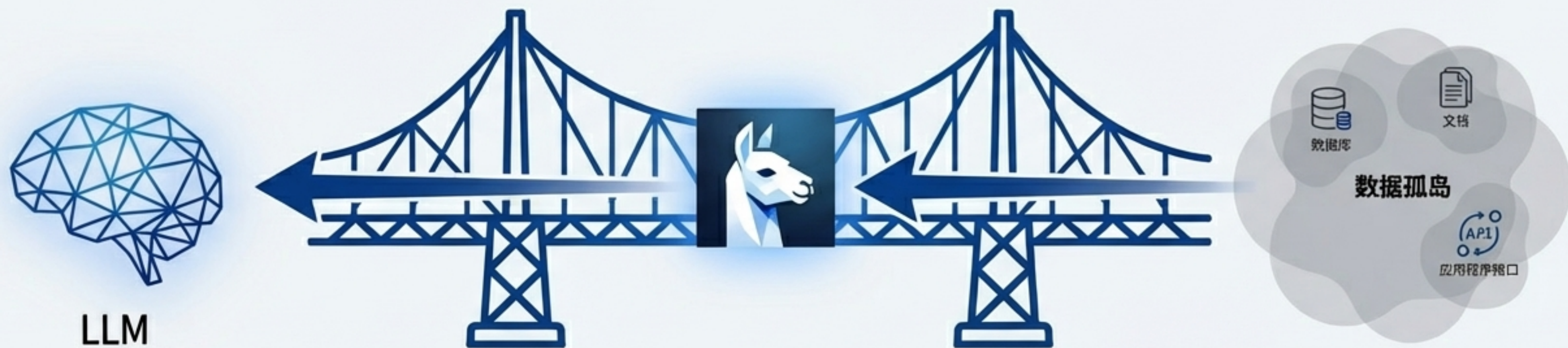
而你的数据， 却存在于孤岛之中

组织的宝贵数据分散在各种内部系统中，形成了一个“数据孤岛”。



LlamaIndex :

连接你的数据与大模型的桥梁



LlamaIndex是一个用于LLM应用的数据框架，专注于注入、结构化并访问私有或特定领域的的数据。

- 它提供了一套工具，简化了从数据接入到构建上下文增强（RAG）应用的完整流程。
- 由Jerry Liu联合创办。

LlamaIndex如何实现这一切？五大核心工具

LlamaIndex通过一套模块化的工具集，为开发者提供了强大的数据处理与交互能力。



Data connectors（数据连接器）：用于数据接入。



Data indexes（数据索引）：用于数据结构化。



Engines（引擎）：用于数据交互。

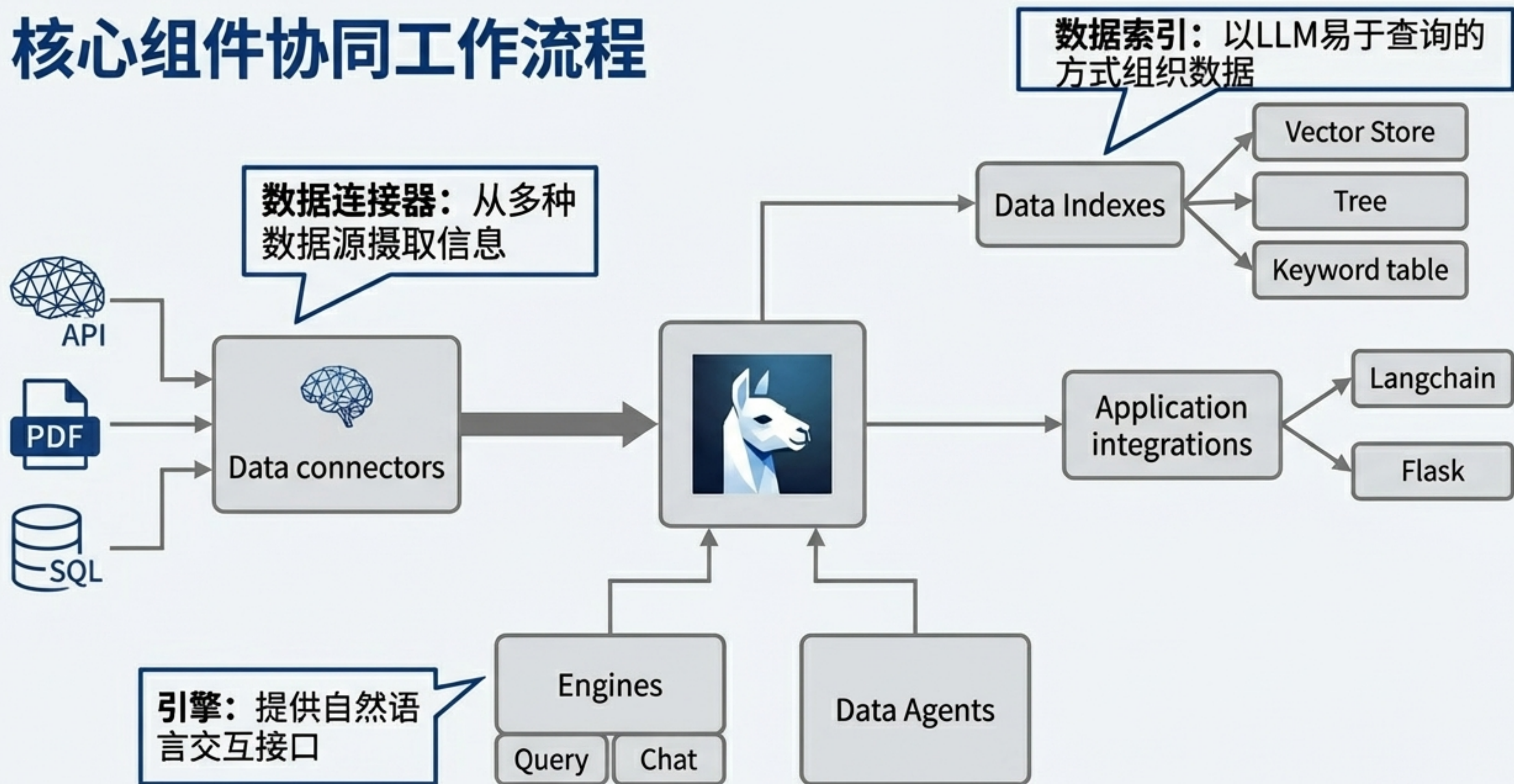


Data agents（数据代理）：更智能的、由LLM驱动的知识工作者。



Application integrations（应用集成）：与外部生态系统无缝衔接。

核心组件协同工作流程



实战：五步构建一个本地知识库问答机器人

我们将使用LlamaIndex，读取一篇本地文章（paul_graham_essay.txt），并让大语言模型能够基于文章内容回答问题。



五大步骤

1. 环境准备与数据加载
2. 模型与服务配置
3. 加载并解析文档
4. 创建数据索引
5. 构建查询引擎并提问

步骤 1 & 2：环境准备与模型配置

安装与导入

```
# 安装 LlamaIndex
%pip install -q llama-index -i https://pypi.doubanio.com/simple

# 导入核心模块
import os
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.deepseek import DeepSeek
```

配置API Key与模型

```
# 设置 API Key (以DeepSeek为例)
os.environ["DEEPSEEK_API_KEY"] = "sk-xxxxxxxxxxxxxxxxxxx"
api_key = os.environ["DEEPSEEK_API_KEY"]

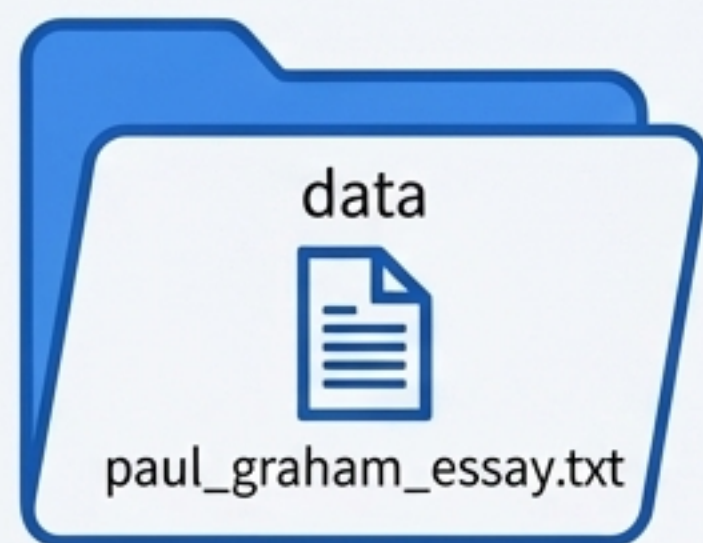
# 配置 LLM 和 Embedding 模型
llm = DeepSeek(model="deepseek-chat", api_key=api_key)
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")

Settings.llm = llm
Settings.embed_model = embed_model
```

配置LLM用于理解和生成文本，
配置Embedding模型用于将文本转换为向量。

步骤 3：加载并解析文档

LlamaIndex仅需一行代码即可从目录中加载所有文档。



```
# 假设 'paul_graham_essay.txt' 文件位于 'data' 文件夹中
documents = SimpleDirectoryReader('data').load_data()
```

- `SimpleDirectoryReader` 会自动识别并读取指定文件夹下的所有支持文件格式。
- `load_data()` 方法将文件内容加载到内存中，形成一个文档对象列表。

步骤 4：创建数据索引

将加载的文档转换为可供LLM高效检索的向量索引。

从文档创建向量存储索引

```
index = VectorStoreIndex.from_documents(documents)
```



步骤 5：构建查询引擎并提问

创建一个查询接口，用自然语言与你的数据进行对话。

```
# 从索引创建查询引擎
query_engine = index.as_query_engine()

# 提出你的问题
response = query_engine.query("who is the author?")
print(response)
```

- ``as_query_engine()`` 是最常用的与数据交互的方式。
- 当你调用 ``query()`` 时，LlamaIndex会将你的问题向量化，在索引中找到最相关的文本块，然后将这些文本块作为上下文连同你的问题一起发送给LLM，最终生成答案。



结果： LLM现在“知道”了你的文档内容

```
response = query_engine.query("who is the author?")  
print(response)
```

The author is Paul Graham.

```
response = query_engine.query("Introduce me Paul Graham")  
print(response)
```

Paul Graham is a programmer, writer, and investor. He is known for his work on Lisp, for co-founding Viaweb (which was acquired by Yahoo and became Yahoo Store), and for co-founding the startup incubator Y Combinator. The essay describes his early experiences with programming, starting from junior high school, and his journey through various projects.

LLM成功地从我们提供的私有文档 `paul\_graham\_essay.txt` 中提取信息并准确回答了问题，而这些信息在其原始训练数据中是不存在的。

总结：用LlamaIndex解锁你的数据潜力

- **我们面临的挑战：**大语言模型无法访问私有数据。
- **我们发现的桥梁：**LlamaIndex提供了连接数据与LLM的完整工具链。
- **我们验证的力量：**通过几行代码，我们成功构建了一个能理解私有文档的问答机器人。

LlamaIndex极大地简化了构建上下文增强（RAG）应用的复杂度，让任何开发者都能轻松地让大语言模型使用私有数据，从而构建更强大、更具个性化的AI应用。

